Module 1 – Introduction



Outline

- What is AI?
- Foundations and History of Artificial Intelligence
- Problem-solving agents
- Searching for Solutions
- Uninformed Search Strategies:
  - Breadth First search
  - Depth First Search

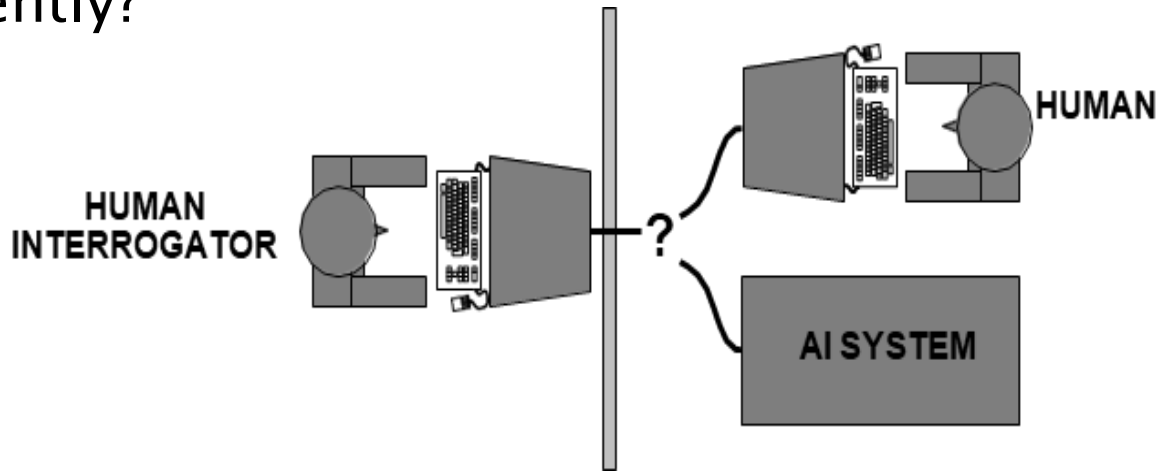
What is AI?

Thinking Humanly	**Thinking Rationally**
"The exciting new effort to make computers think . . . *machines with minds*, in the full and literal sense." (Haugeland, 1985) "[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning . . ." (Bellman, 1978)	"The study of mental faculties through the use of computational models." (Charniak and McDermott, 1985) "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992)
Acting Humanly	**Acting Rationally**
"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990) "The study of how to make computers do things at which, at the moment, people are better." (Rich and Knight, 1991)	"Computational Intelligence is the study of the design of intelligent agents." (Poole *et al.*, 1998) "AI . . . is concerned with intelligent behavior in artifacts." (Nilsson, 1998)

Figure 1.1 Some definitions of artificial intelligence, organized into four categories.

Acting humanly: The Turing Test approach

- Turing Test- proposed by Alan Turing (1950)
- Can machine think?→ "Can machine behave Intelligently?"



- It's a test for intelligent behavior.
- A machine might have 30% chance of providing response like a human.

- Programming a computer to pass rigorously for the questions posed.
- Computer need to possess the following capabilities:
  - **Natural language processing** to enable it to communicate successfully in English.
  - **Knowledge representation** to store what it knows or hears.
  - **Automated reasoning** to use the stored information to answer questions and to draw new conclusions
  - **Machine learning** to adapt to new circumstances and to detect and extrapolate patterns.
  - **Computer vision** to perceive objects, and
  - **Robotics** to manipulate objects and move about.

- These six disciplines compose most of AI, and Turing deserves credit for designing a test that remains relevant 60 years later.

Thinking humanly: The cognitive modeling approach

- To claim a program thinks like a human, understanding human thinking is crucial and challenging.

- Three methods for understanding human thinking:
  - Introspection: Attempting to catch one's own thoughts.
  - Psychological experiments: Observing individuals in action.
  - Brain imaging: Observing the brain in action.

- Emphasis on the need to delve into the workings of the human mind.

Translating Theory into Computer Programs:

- Importance of a precise theory of the mind for creating a computer program.
- Linking theory to program: If program input-output matches human behavior, it suggests shared mechanisms.
- Example: Allen Newell and Herbert Simon's developed GPS.
- Focus on comparing program reasoning steps to traces of human subjects solving the same problems.

- **Overview of Cognitive Science**:
  - Integration of computer models from AI and experimental techniques from psychology.

- Recognition of the complexity of cognitive science, worthy of extensive study.
- Actual cognitive science based on experimental investigation of humans or animals.
- Continued collaboration between AI and cognitive science, notably in areas like computer vision integrating neurophysiological evidence.

Thinking rationally: The “laws of thought” approach

- Aristotle is one of the first to codify "right thinking" and irrefutable reasoning processes.
- Example of Aristotle's syllogisms as patterns for argument structures, ensuring correct conclusions with correct premises.
- The emergence of these "laws of thought" governed the mind, leading to the field of logic.
- Mention of 19th-century logicians developing precise notation for statements about objects and their relations.

Logicist Tradition in AI

- Overview of the logicist tradition in artificial intelligence, building on precise logical notation.
- Development of programs by 1965 capable of solving any solvable problem described in logical notation.
- Two main obstacles to this approach:
  - Difficulty in formalizing informal knowledge in logical terms, especially when knowledge is uncertain.
  - The gap between solving a problem "in principle" and solving it in practice.
- Specific challenges in the logicist tradition, including the need for formalization and practical problem-solving guidance.

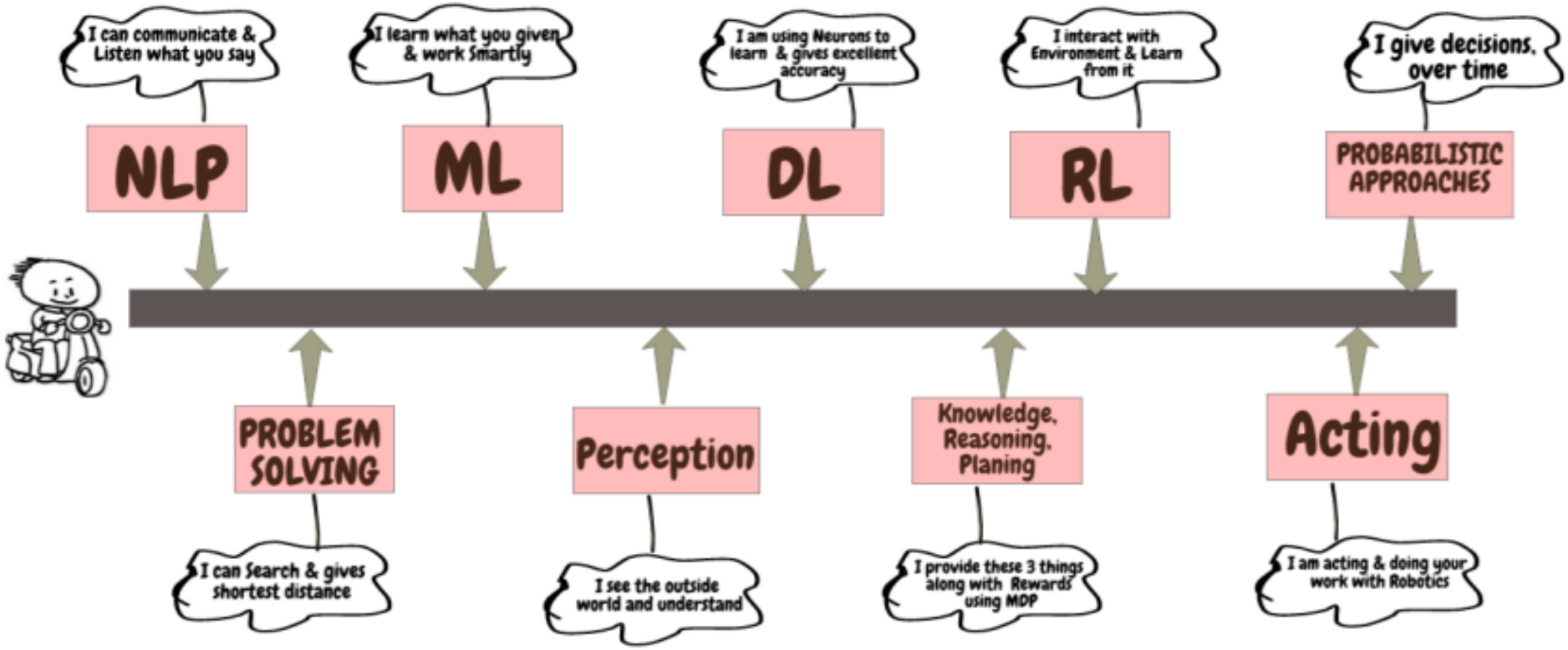
Acting rationally: The rational agent approach

- An agent is just Something that acts (derived from the Latin agere, to do).
- Characteristics of computer agents are Autonomy, perception, persistence, adaptability, and goal creation.
- Rational agent is one that acts to achieve the best outcome.
- Relationship with correct inferences: Correct inferences are part of rationality, but not the entirety.
- Examples of rational actions: Logical reasoning and reflex actions like recoiling from a hot stove.
- Connection to the Turing Test: Skills needed for the Turing Test align with acting rationally.

- Advantages of the rational-agent approach:
  - More general than the "laws of thought" approach, encompassing various mechanisms for achieving rationality.
  - More amenable to scientific development than approaches based on human behavior or thought.

- The standard of rationality is mathematically well-defined and generalizable.
- Challenges in achieving perfect rationality in complicated environments due to high computational demands.
- Adopt the working hypothesis that perfect rationality as a starting point for analysis, acknowledging the issue of limited rationality.

Fields in AI


I can communicate & Listen what you say
NLP
I learn what you given & work Smartly
ML
I am using Neurons to learn & gives excellent accuracy
DL
I interact with Environment & Learn from it
RL
I give decisions, over time
PROBABILISTIC APPROACHES
PROBLEM SOLVING
I can Search & gives shortest distance
Perception
I see the outside world and understand
Knowledge, Reasoning, Planing
I provide these 3 things along with Rewards using MDP
Acting
I am acting & doing your work with Robotics

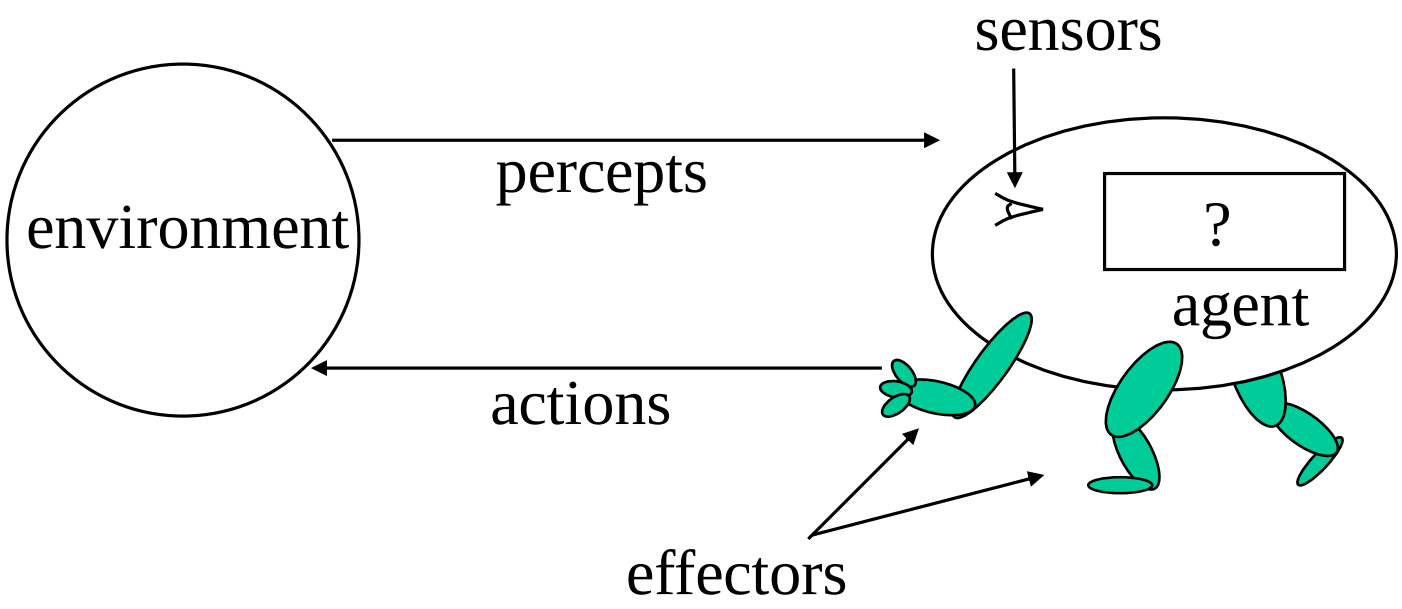
Applications

- Robotic Vehicles
- Speech recognition
- Autonomous planning and scheduling
- Game Playing
- Spam filtering
- Logistics
- Machine Translation

Module 1: Intelligent Agents

Nature of agents, diversity of environments and agent types

An agent and its environment



How to design an intelligent agent?

- **Definition**: An *agent* perceives its environment via sensors and acts in that environment with its effectors.

  Hence, an agent gets percepts one at a time, and maps this percept sequence to actions (one action at a time)

- **Properties**:
  - Autonomous
  - Interacts with other agents plus the environment
  - Reactive to the environment
  - Pro-active (goal-directed)

- *Percept*: The term percept to refer to the agent's perceptual inputs at any given instant
- *Agent's percept sequence* is the complete history of everything the agent has ever perceived
- ***An agent's choice of action at any given instant can depend on the entire percept sequence observed to date, but not on anything it hasn't perceived***
- *Agent function:* maps any given percept sequence to an action
- *Agent program*: runs on the physical architecture to produce *f*, the agent function for an artificial agent will be implemented by an agent program

The Vacuum Cleaner World

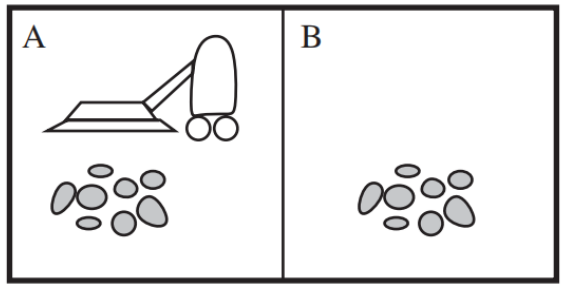


Figure 2.2 A vacuum-cleaner world with just two locations.

- This particular world has just two locations: A & B
- The vacuum agent perceives which square it is in and whether there is dirt in the square.
- It can choose to move left, move right, suck up the dirt, or do nothing.

Percepts: location and contents[A,Dirty]

Actions:*Left*, *Right*, *Suck*, *NoOp*

Agent's function-> look-up table

Percept sequence	Action
$[A, Clean]$	$Right$
$[A, Dirty]$	$Suck$
$[B, Clean]$	$Left$
$[B, Dirty]$	$Suck$
$[A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Dirty]$	$Suck$
⋮	⋮
$[A, Clean], [A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Clean], [A, Dirty]$	$Suck$
⋮	⋮

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

function REFLEX-VACUUM-AGENT([*location*,*status*]) **returns** an action

if *status* = *Dirty* **then return** *Suck*
else if *location* = *A* **then return** *Right*
else if *location* = *B* **then return** *Left*

Figure 2.8 The agent program for a simple reflex agent in the two-state vacuum environment. This program implements the agent function tabulated in Figure 2.3.

Good Behaviour: Concept of Rationality

- The concept of **rationality** in an agent's behavior refers to its ability to make decisions that maximize its performance based on its goals and the available information. Eg: SIRI, Alexa
- A **rational agent** is one that does the right thing.
- What does it mean to do the right thing?
- Considering the *consequences* of the agent's behavior
- When an agent is plunked down in an environment, it generates a sequence of actions according to the percepts it receives.
- Eg: The assistant schedules the meeting for a time that works for everyone, minimizing conflicts and maximizing convenience. It may also notify the user if no ideal times are available, proposing the next best option based on priorities.

Performance measure

- An *objective criterion for success* of an agent's behavior. Evaluates any given sequence of environment states.
- **Performance measures of a vacuum**-cleaner agent: amount of dirt cleaned up, amount of time taken, amount of electricity consumed, level of noise generated, etc.
- **Performance measures self**-driving car: time to reach destination (minimize), safety, predictability of behavior for other agents, reliability, etc.
- Performance measure of game-playing agent: win/loss percentage (maximize), robustness, unpredictability (to “confuse” opponent), etc.

Rationality

- What is rational at any given time depends on four things:
    - Performance measuring success
    - Agents prior knowledge of environment
    - Actions that agent can perform
    - Agent's percept sequence to date

- Rational Agent**: For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.**

Automated taxi driving system

- **Percepts**: Video, sonar, speedometer, odometer, engine sensors, keyboard input, microphone, GPS, …
- **Actions**: Steer, accelerate, brake, horn, speak/display, …
- **Goals**: Maintain safety, reach destination, maximize profits (fuel, tire wear), obey laws, provide passenger comfort, …
- **Environment**: U.S. urban streets, freeways, traffic, pedestrians, weather, customers, …
- **Different aspects of driving may require different types of agent programs!**

Examples of agents in different types of applications

Agent type	Percepts	Actions	Goals	Environment
Medical diagnosis system	Symptoms, findings, patient's answers	Questions, tests, treatments	Healthy patients, minimize costs	Patient, hospital
Satellite image analysis system				
Part-picking robot				
Refinery controller				
Interactive English tutor				

Examples of agents in different types of applications

Agent type	Percepts	Actions	Goals	Environment
Medical diagnosis system	Symptoms, findings, patient's answers	Questions, tests, treatments	Healthy patients, minimize costs	Patient, hospital
Satellite image analysis system	Pixels of varying intensity, color	Print a categorization of scene	Correct categorization	Images from orbiting satellite
Part-picking robot				
Refinery controller				
Interactive English tutor				

Examples of agents in different types of applications

Agent type	Percepts	Actions	Goals	Environment
Medical diagnosis system	Symptoms, findings, patient's answers	Questions, tests, treatments	Healthy patients, minimize costs	Patient, hospital
Satellite image analysis system	Pixels of varying intensity, color	Print a categorization of scene	Correct categorization	Images from orbiting satellite
Part-picking robot	Pixels of varying intensity	Pick up parts and sort into bins	Place parts in correct bins	Conveyor belts with parts
Refinery controller				
Interactive English tutor				

Examples of agents in different types of applications

Agent type	Percepts	Actions	Goals	Environment
Medical diagnosis system	Symptoms, findings, patient's answers	Questions, tests, treatments	Healthy patients, minimize costs	Patient, hospital
Satellite image analysis system	Pixels of varying intensity, color	Print a categorization of scene	Correct categorization	Images from orbiting satellite
Part-picking robot	Pixels of varying intensity	Pick up parts and sort into bins	Place parts in correct bins	Conveyor belts with parts
Refinery controller	Temperature, pressure readings	Open, close valves; adjust temperature	Maximize purity, yield, safety	Refinery
Interactive English tutor				

Examples of agents in different types of applications

Agent type	Percepts	Actions	Goals	Environment
Medical diagnosis system	Symptoms, findings, patient's answers	Questions, tests, treatments	Healthy patients, minimize costs	Patient, hospital
Satellite image analysis system	Pixels of varying intensity, color	Print a categorization of scene	Correct categorization	Images from orbiting satellite
Part-picking robot	Pixels of varying intensity	Pick up parts and sort into bins	Place parts in correct bins	Conveyor belts with parts
Refinery controller	Temperature, pressure readings	Open, close valves; adjust temperature	Maximize purity, yield, safety	Refinery
Interactive English tutor	Typed words	Print exercises, suggestions, corrections	Maximize student's score on test	Set of students

Agents and Environments

- An agent perceives its environment through sensors
  - the complete set of inputs at a given time is called a percept
  - the current percept, or a sequence of percepts may influence the actions of an agent
- it can change the environment through actuators
  - an operation involving an actuator is called an action
  - actions can be grouped into action sequences

A Windshield Wiper Agent

How do we design a agent that can wipe the windshields

when needed?

- Goals?
- Percepts?
- Sensors?
- Effectors?
- Actions?
- Environment?

A Windshield Wiper Agent (Cont'd)

- Goals: Keep windshields clean & maintain visibility
- Percepts: Raining, Dirty
- Sensors: Camera (moist sensor)
- Effectors: Wipers (left, right, back)
- Actions: Off, Slow, Medium, Fast
- Environment: Inner city, freeways, highways, weather …

Characteristics of Rational agents

Rationality is distinct from omniscience ("all knowing"). We can behave rationally even when faced with incomplete information.

Agents can perform actions in order to modify future percepts so as to obtain useful information: information gathering, exploration.

An agent is autonomous if its behavior is determined by its own experience (with ability to learn and adapt).

Omniscience

- Omniscient agent
  - Knows the actual outcome of its actions
  - What information would a chess player need to have to be omniscient?
- a rational agent is not omniscient
  - it doesn't know the actual outcome of its actions
  - it may not know certain aspects of its environment
  - A rational agent should do the right thing based on the knowledge it has
- Rationality maximizes expected performance, while perfection maximizes actual performance.

Autonomy

- "Independence"
- A system is autonomous if its behavior is determined by its own experience
  - An alarm that goes off at a prespecified time is not autonomous
  - An alarm that goes off when smoke is sensed is autonomous
- A system without autonomy lacks flexibility

Autonomy

- A system is autonomous to the extent that its own behavior is determined by its own experience.
- Therefore, a system is not autonomous if it is guided by its designer according to a priori decisions.
- To survive, agents must have:
  - Enough built-in knowledge to survive.
  - The ability to learn.

- **Example (Rational Agent):**
- Imagine an autonomous delivery drone tasked with delivering packages efficiently. It calculates the shortest path based on current traffic conditions, weather, and battery life. The decision it makes is rational because it maximizes efficiency given the constraints of the environment.

- **Example (Omniscient Agent):**
- AI tasked with playing chess, but this AI has omniscient capabilities—it knows the position of all pieces, every possible future move both it and its opponent could make, and the outcome of each scenario. With this complete knowledge, it can always make the optimal move and never lose.

Characterizing a Task Environment

Must first specify the setting for intelligent agent design.

PEAS: Performance measure, Environment, Actuators, Sensors

Example: the task of designing a self-driving car

Performance measure Safe, fast, legal, comfortable trip

- Environment Roads, other traffic, pedestrians
- Actuators Steering wheel, accelerator, brake, signal, horn
- Sensors Cameras, LIDAR (light/radar), speedometer, GPS, odometer engine sensors, keyboard

PEAS Description of Task Environments

used for high-level characterization of agents

Performance Measures	used to evaluate how well an agent solves the task at hand
Environment	surroundings beyond the control of the agent
Actuators	determine the actions the agent can perform
Sensors	provide information about the current state of the environment

Examples of agent types and their PEAS description

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Figure 2.5 Examples of agent types and their PEAS descriptions.

Properties of Task Environments

1) Fully observable / Partially observable

– **If an agent's sensors give it access to the complete state of the environment needed to choose an action, the environment is fully observable.**
(e.g. chess)



Also termed **Accessible vs. Inaccessible**

Such environments are convenient, since the agent is freed from the task of keeping track of the changes in the environment

- Driving in heavy fog (you can’t see the whole road ahead).
- Medical diagnosis (you don’t directly observe the internal state of a patient, only symptoms and test results).
- Robot navigation in an unfamiliar building without a complete map.

2) Deterministic / Stochastic

- An environment is **deterministic** If the next state of the environment is completely determined by the current state of the environment and the action of the agent;
- changes in the environment are predictable

- In a **stochastic** environment, there are multiple, unpredictable outcomes. (If the environment is deterministic except for the actions of other agents, then the environment is **strategic**).

3) Episodic / Sequential

- In an episodic environment, the agent's experience is divided into atomic episodes. Each episode consists of the agent perceiving and then performing a single action.

- Subsequent episodes do not depend on what actions occurred in previous episodes. Choice of action in each episode depends only on the episode itself.
  (E.g., classifying images.)

- In a sequential environment, the agent engages in a series of connected episodes. Current decision can affect future decisions. (E.g., chess and driving)

4) Static / Dynamic

- A static environment does not change while the agent is thinking.
- The passage of time as an agent deliberates is irrelevant.
  - Example: Crossword puzzle – the puzzle doesn't change while you think about the next word.

- Dynamic environment – The environment can change while the agent is thinking or before it acts.
  - Example: Soccer match – other players keep moving, and the ball changes position while you decide your next move.

- The environment is semidynamic if the environment itself does not change with the passage of time but the agent's performance score does.
  - Example: Timed exam – the questions don't change, but your score potential decreases as time runs out.

5) Discrete / Continuous

- If the number of distinct percepts and actions is limited, the environment is discrete, otherwise it is continuous
  - Ex. For discrete-chess
- Continuous environment – Percepts and actions can vary smoothly over a range of values.
  - Example: Taxi driving – steering angle, speed, and passenger locations vary continuously

6) Single agent / Multi-agent

- If the environment contains other intelligent agents, the agent needs to be concerned about strategic, game-theoretic aspects of the environment (for either cooperative *or* competitive agents)

  Ex: Solving crossword puzzle – single agent

  Chess – two agent

- Multi-agent environment: The environment contains multiple intelligent agents whose actions can affect each other. These may be:
  - Cooperative – agents work together toward a shared goal.
  - Competitive – agents have opposing objectives.
- Most engineering environments don't have multi-agent properties, whereas most social and economic systems get their complexity from the interactions of (more or less) rational agents.

Examples of task environments

Task Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Crossword puzzle	Fully	Deterministic	Sequential	Static	Discrete	Single
Chess with a clock	Fully	Strategic	Sequential	Semi	Discrete	Multi
Poker	Partially	Strategic	Sequential	Static	Discrete	Multi
Backgammon	Fully	Stochastic	Sequential	Static	Discrete	Multi
Taxi driving	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
Medical diagnosis	Partially	Stochastic	Sequential	Dynamic	Continuous	Single
Image-analysis	Fully	Deterministic	Episodic	Semi	Continuous	Single
Part-picking robot	Partially	Stochastic	Episodic	Dynamic	Continuous	Single
Refinery controller	Partially	Stochastic	Sequential	Dynamic	Continuous	Single
Interactive English tutor	Partially	Stochastic	Sequential	Dynamic	Discrete	Multi

Figure 2.6 Examples of task environments and their characteristics.

Characteristics of environments

	Observable	Deterministic	Episodic	Static	Discrete
Solitaire					
Part-Picking Robot					
Taxi driving					
Internet shopping					
Medical diagnosis					

Characteristics of environments

	Observable	Deterministic	Episodic	Static	Discrete
Solitaire	**No**	**Yes**	**Yes**	**Yes**	**Yes**
Part-Picking Robot					
Taxi driving					
Internet shopping					
Medical diagnosis					

Characteristics of environments

	Observable	Deterministic	Episodic	Static	Discrete
Solitaire	No	Yes	Yes	Yes	Yes
Part-Picking Robot	**No**	**No**	**Yes**	**No**	**No**
Taxi driving					
Internet shopping					
Medical diagnosis					

Characteristics of environments

	Observable	Deterministic	Episodic	Static	Discrete
Solitaire	No	Yes	Yes	Yes	Yes
Part-Picking Robot	No	No	Yes	No	No
Taxi driving	**No**	**No**	**No**	**No**	**No**
Internet shopping					
Medical diagnosis					

Characteristics of environments

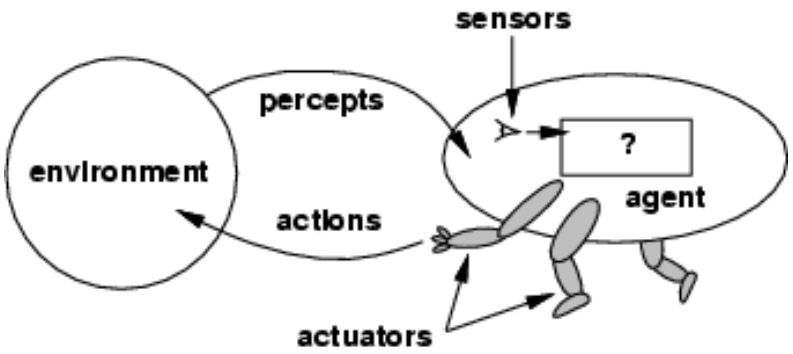
	Observable	Deterministic	Episodic	Static	Discrete
Solitaire	No	Yes	Yes	Yes	Yes
Part-Picking Robot	No	No	Yes	No	No
Taxi driving	No	No	No	No	No
Internet shopping	**No**	**No**	**No**	**No**	**No**
Medical diagnosis					

Characteristics of environments

	Observable	Deterministic	Episodic	Static	Discrete
Solitaire	No	Yes	Yes	Yes	Yes
Part-Picking Robot	No	No	Yes	No	No
Taxi driving	No	No	No	No	No
Internet shopping	No	No	No	No	No
Medical diagnosis	**No**	**No**	**No**	**No**	**No**

→ Lots of real-world domains fall into the hardest case!

Agent functions and programs

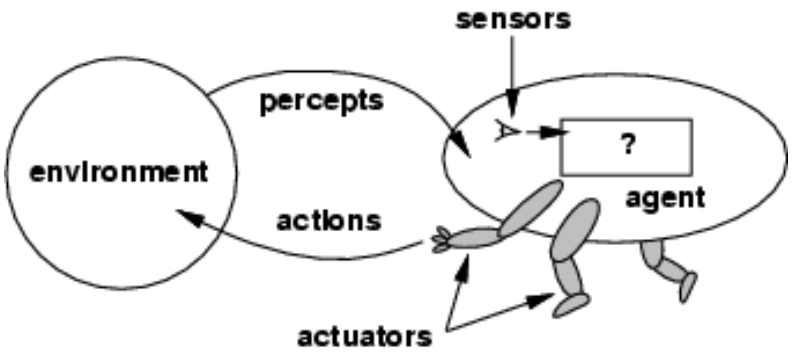


The agent function maps from percept histories to actions

$$f: \mathcal{P}^* \rightarrow \mathcal{A}$$

The job of AI is to design the agent program that implements the agent function mapping percepts to actions

Agent functions and programs



We assume this program will run on some sort of computing device with physical sensors n actuators

The agent program runs (internally) on the physical architecture to produce *f*

agent = architecture + program

Aim: Design an agent program assuming an architecture that will make the percepts from the sensors available to the program.

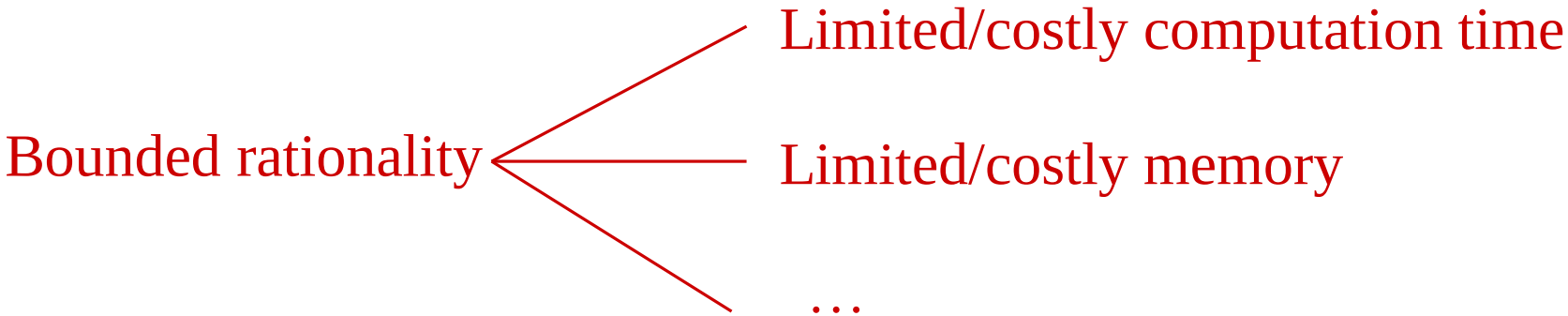
Definition of ideal rational agent

Ideal Rational Agent: For each possible percept sequence, such an agent does whatever action is expected to maximize its performance measure, on the basis of the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

What do you think? Is this an acceptable definition?

Not looking left when crossing the street: If I don't see a car coming from the left, it is rational to cross the street?

No. Should also consider taking *information gathering actions*.



Bounded rationality

- Limited or costly computation time
  - The agent may not have enough processing power or time to compute the optimal solution.
  - Example: A chess AI in a timed match may use an approximate strategy rather than evaluating every possible move.
- Limited or costly memory
  - The agent cannot store or recall all possible states or rules.
  - Example: A navigation robot with limited storage may keep only part of the map in memory at once.

Agent's strategy

- Agent's *strategy* is a mapping from percept sequence to action
- How to encode an agent's strategy?
  - Long list of what should be done for each possible percept sequence
  - vs. shorter specification (e.g. algorithm)

Skeleton agent

function SKELETON-AGENT (*percept*) **returns** action
  static: *memory*, the agent's memory of the world

  memory ← UPDATE-MEMORY(*memory*,*percept*)
  action ← CHOOSE-BEST-ACTION(*memory*)
  memory ← UPDATE-MEMORY(*memory*, *action*)
  return *action*

On each invocation, the agent's memory is updated to reflect the new percept, the best action is chosen, and the fact that the action was taken is also stored in the memory. The memory persists from one invocation to the next.

Input = Percept, not history
NOTE: Performance measure is not part of the agent

Examples of how the agent function can be implemented

More sophisticated

1. Table-driven agent
2. Simple reflex agent
3. Reflex agent with internal state
4. Agent with explicit goals
5. Utility-based agent

1. Table-driven agent

function TABLE-DRIVEN-AGENT (percept) **returns** action
  static: *percepts*, a sequence, initially empty
    table, a table, indexed by percept sequences, initially fully specified

  append *percept* to the end of *percepts*
  action ← LOOKUP(*percepts*, *table*)
  return *action*

An agent based on a prespecified lookup table. It keeps track of percept sequence and just looks up the best action

- **Problems**

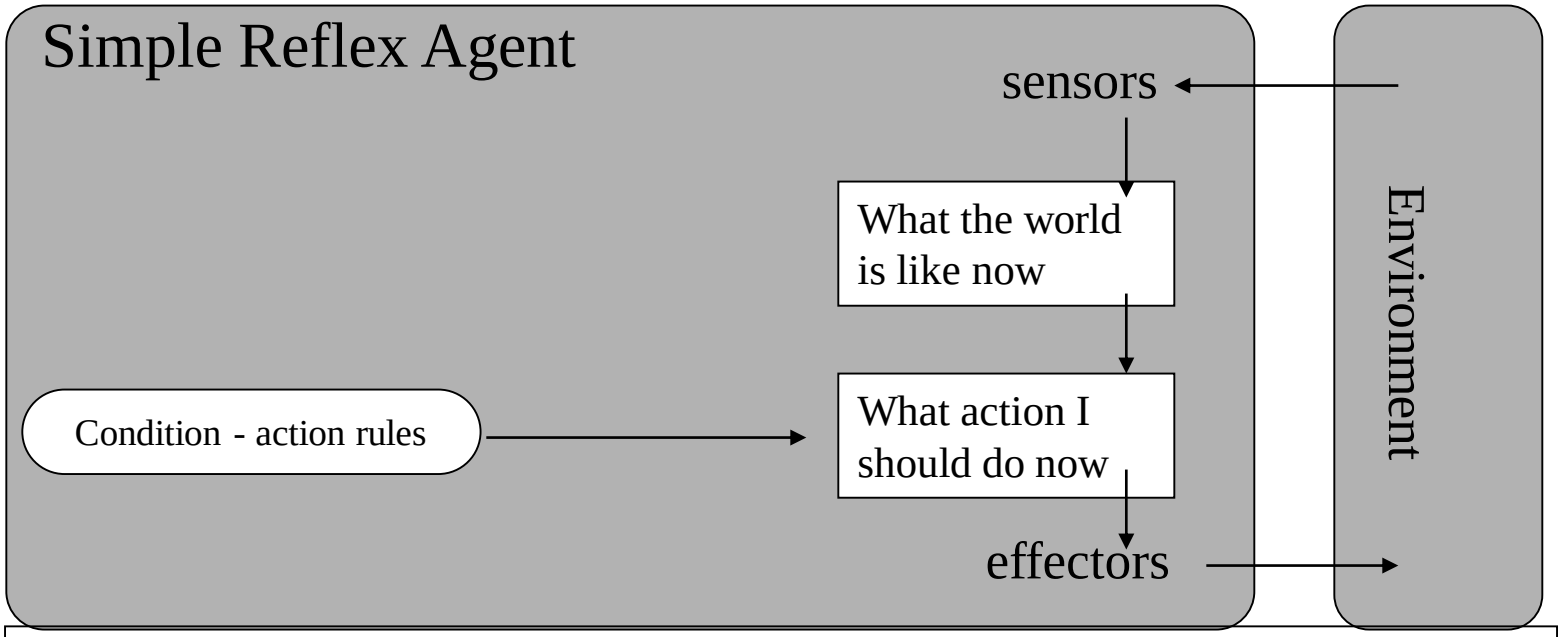
  – Huge number of possible percepts, Too big to generate and to store (Chess has about 10^120 states, for example) => lookup table would be huge

  – Takes long time to build the table

  – Not adaptive to changes in the environment; requires entire table to be updated if changes occur

2. Simple reflex agent

- Differs from the lookup table based agent is that the condition (that determines the action) is already higher-level interpretation of the percepts
  - Percepts could be e.g. the pixels on the camera of the automated taxi
- A simple reflex agent acts only on the current percept (situation), ignoring the history of past percepts.



function SIMPLE-REFLEX-AGENT(*percept*) **returns** action
static: *rules*, a set of condition-action rules

state ← INTERPRET-INPUT (*percept*)
rule ← RULE-MATCH (*state*,*rules*)
action ← RULE-ACTION [*rule*]
return *action*

First match.
No further matches sought.
Only one level of deduction.

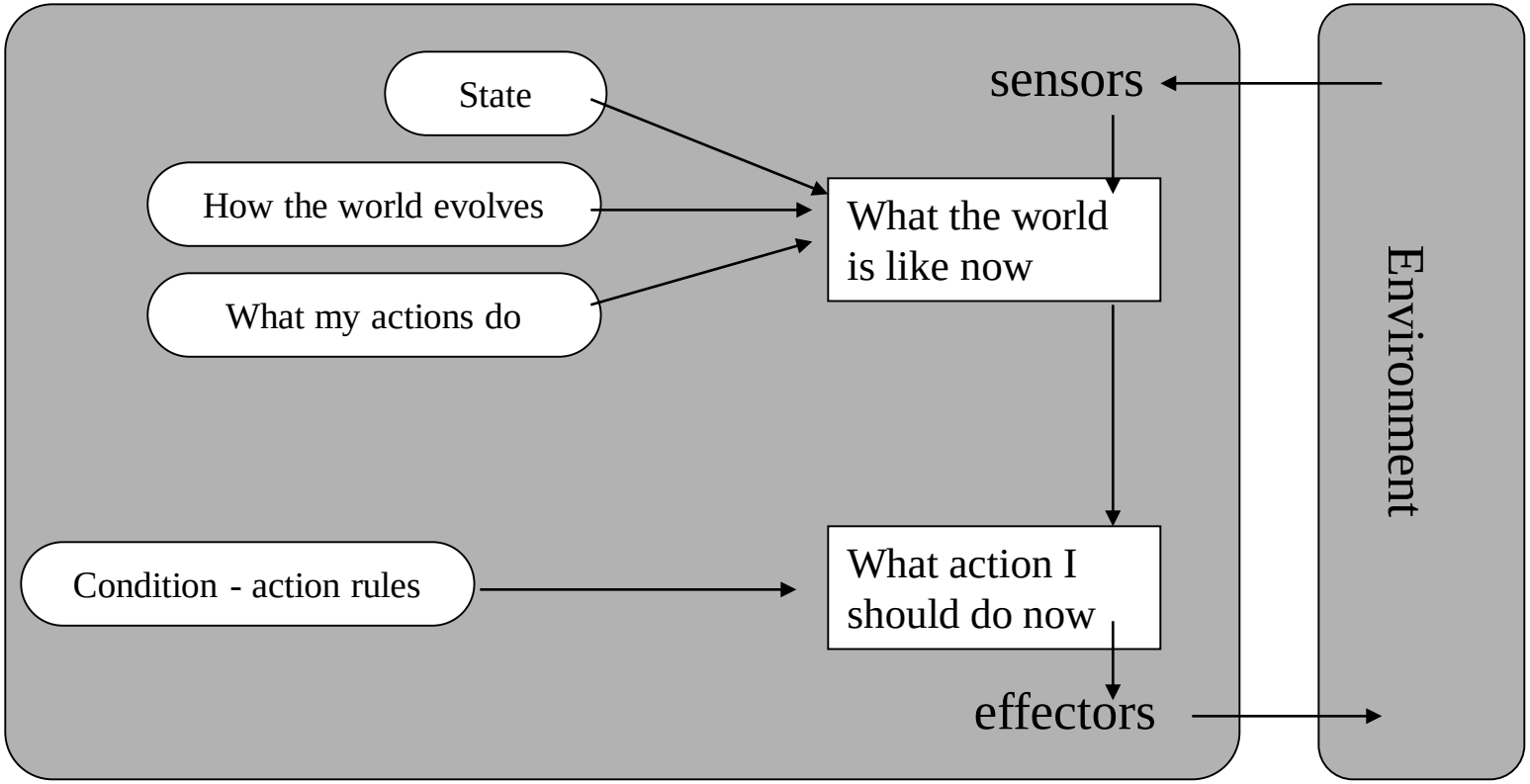
A simple reflex agent works by finding a rule whose condition matches the current situation (as defined by the percept) and then doing the action associated with that rule.

Simple reflex agent…

- **Table lookup of condition-action pairs defining all possible condition-action rules necessary to interact in an environment**
  - e.g. if car-in-front-is-breaking then initiate breaking
- Example: Automatic Door
- Condition: If sensor detects someone near the door
- Action: Open door
- Condition: If no one near
- Action: Close door
- **Problems**
  - Table is still too big to generate and to store (e.g. taxi)
  - Takes long time to build the table
  - No knowledge of non-perceptual parts of the current state
  - Looping: Can’t make actions conditional

- **Condition-action rule**: Thermostat
  - **Condition**: Temperature < 20°C → **Action**: Turn on heater.
  - **Condition**: Temperature ≥ 20°C → **Action**: Turn off heater.
- **Condition-action rule**: Automatic Door System
  - **Condition**: Person detected → **Action**: Open door.
  - **Condition**: No person detected → **Action**: Close door.
- **Condition-action rule**: Light Sensors
  - **Condition**: Light level < threshold → **Action**: Turn on lights.
  - **Condition**: Light level ≥ threshold → **Action**: Turn off lights.

3. Model- based Reflex agent



- **Sensors:** Sensors serve as the interface between the agent and its surroundings, gathering information on the environment's current state..
- **Internal Model:** The internal model is the agent's understanding of the environment, encompassing knowledge of dynamics, rules, and potential outcomes of actions.
- **Reasoning Component:** The reasoning component uses information from sensors and internal models to make decisions. **Actuators:** Actuators facilitate the agent's interaction with the environment through executing actions, whether physical like motors or virtual interfaces

Model- based Reflex agent

- Unlike simple reflex agents that only react to current perceptual information, model-based reflex agents maintain an internal representation, or model, of the environment that allows them to anticipate the consequences of their actions.
- Model-based reflex agents are a type of intelligent agent designed to interact with partial observable or dynamic environments.

Model- based Reflex agent …

function REFLEX-AGENT-WITH-STATE (*percept*) **returns** action
static: *state*, a description of the current world state
rules, a set of condition-action rules

state ← UPDATE-STATE (*state*, *percept*)
rule ← RULE-MATCH (*state*, *rules*)
action ← RULE-ACTION [*rule*]
state ← UPDATE-STATE (*state*, *action*)
return *action*

A reflex agent with internal state works by finding a rule whose condition matches the current situation (as defined by the percept and the stored internal state) and then doing the action associated with that rule.

- **Condition-action rule**: Self Driving Car
  - **Condition**: Car ahead slows down → **Action**: Gradually brake based on traffic context.
  - **Condition**: Traffic light ahead turns red → **Action**: Prepare to stop.
- **Condition-action rule**: Robotic Vacuum Cleaner
  - **Condition**: Area already cleaned → **Action**: Skip area and move to a new location.
  - **Condition**: Low battery detected → **Action**: Navigate back to the charging dock.
- **Condition-action rule**: Autonomous Warehouse Robots
  - **Condition**: Goods need to be picked from location A → **Action**: Move to A and avoid obstacles using internal map.
  - **Condition**: Path blocked → **Action**: Recalculate an optimal route using the model.

Model- based Reflex agent …

- Encode "internal state of the world to remember the past as contained in earlier percepts

- Needed because sensors do no usually give the entire state of the world at each input, so perception of the environment is captured over time. "State" used to encode different "world states" that generate the same immediate percept

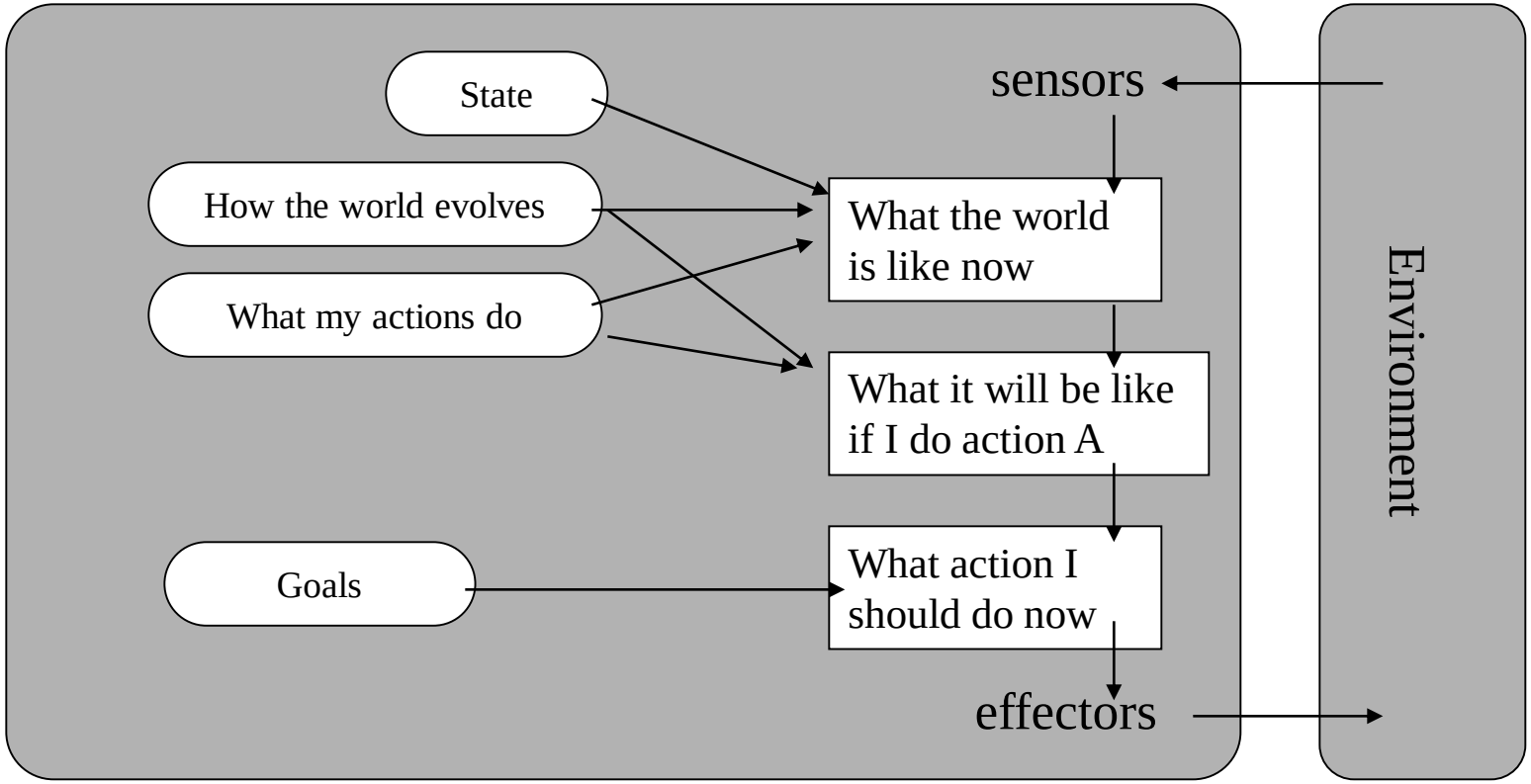
- Requires ability to represent change in the world with/without the agent
  - one possibility is to represent just the latest state, but then cannot reason about hypothetical courses of action

- **Example:** Rodney Brook's *Subsumption Architecture*. Main idea: build complex intelligent robots by decomposing behaviors into a hierarchy of skills, each completely defining a complete percept-action cycle for one very specific task. For example, avoiding contact, wandering, exploring, recognizing doorways, etc. Each behavior is modeled by a finite-state machine with a few states (though each state may correspond to a complex function or module). Behaviors are loosely-coupled, asynchronous interactions

4. Goal Based Agents

- Unlike simple reflex agents that act solely based on current perceptions, goal-based agents consider future consequences of their actions, ensuring that they align with the set objectives.

4. Goal Based Agents



Goal Based Agents

- Choose actions so as to achieve a (given or computed) goal = a description of desirable situations. e.g. where the taxi wants to go

- Keeping track of the current state is often not enough – need to add goals to decide which situations are good

- Deliberative instead of reactive

- May have to consider long sequences of possible actions before deciding if goal is achieved – involves considerations of the future, “what will happen if I do…?” (search and planning)

- More flexible than reflex agent. (e.g. rain / new destination) In the reflex agent, the entire database of rules would have to be rewritten

Route Planning in GPS Systems

• **Goal**: Get the user from point A to point B in the most efficient way (e.g., shortest route or fastest route).

•**Decision process**:

•Evaluates various routes based on distance, traffic conditions, and road types.

•Takes action (guides the user) to follow the selected optimal route.

Chess-playing AI

- **Goal**: Checkmate the opponent.
- **Decision process**: Evaluates various possible moves,

predicting future board states and assessing how each move helps achieve the goal of checkmate.

- Chooses the move that maximizes the chances of winning.

Personal Health Monitoring System

•**Goal**: Maintain or improve health metrics
(e.g., target heart rate, sleep duration).

•**Decision process**:

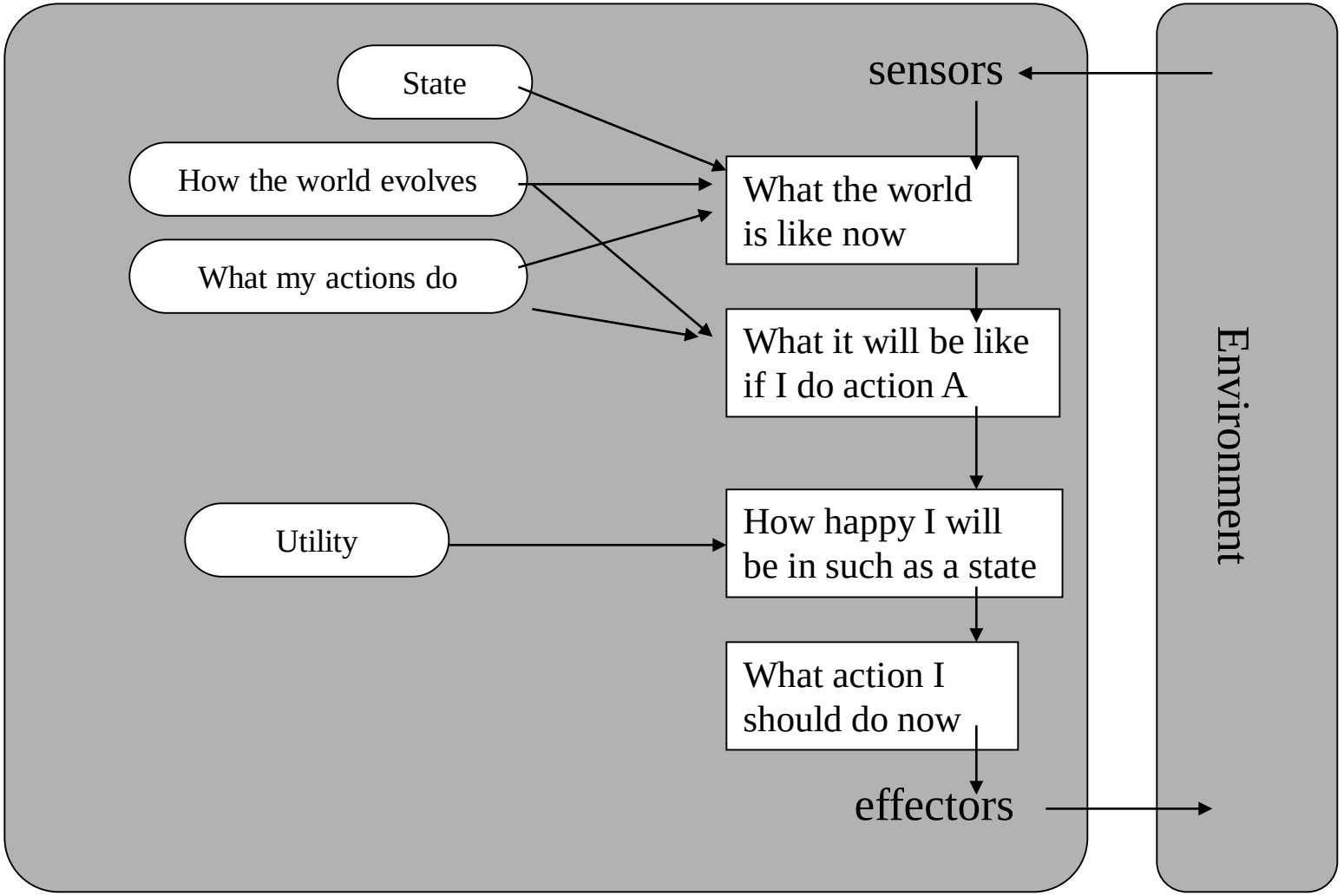
•Analyzes real-time data from sensors (heart rate, steps taken, etc.).

•Suggests actions to the user
(e.g., walk more, get more sleep) to meet health goals.

5. Utility-based agent

- These agents are similar to the goal-based agent but provide an extra component of utility measurement which makes them different by providing a measure of success at a given state.
- Utility-based agent act based not only goals but also the best way to achieve the goal.
- The Utility-based agent is useful when there are multiple possible alternatives, and an agent has to choose in order to perform the best action.
- The utility function maps each state to a real number to check how efficiently each action achieves the goals.

5. Utility-based agent



Utility-based agent …

• When there are multiple possible alternatives, how to decide which one is best?

• A goal specifies a crude destination between a happy and unhappy state, but often need a more general performance measure that describes "degree of happiness"

• Utility function U: State → Reals indicating a measure of success or happiness when at a given state

• Allows decisions comparing choice between conflicting goals, and choice between likelihood of success and importance of goal (if achievement is uncertain)

- **Financial Trading Algorithms**
- **How it works**: Automated trading systems make buying or selling decisions based on a combination of factors such as market conditions, risk tolerance, and potential returns.
  Utility factors:
  - Profit potential (maximizing financial return).
  - Risk (minimizing potential losses).
  - Market volatility.
- **Decision process**:
  - The agent evaluates different trading options, assigning utility values based on expected profit and risk.
  - It selects the trade that maximizes its utility function, balancing risk and reward.

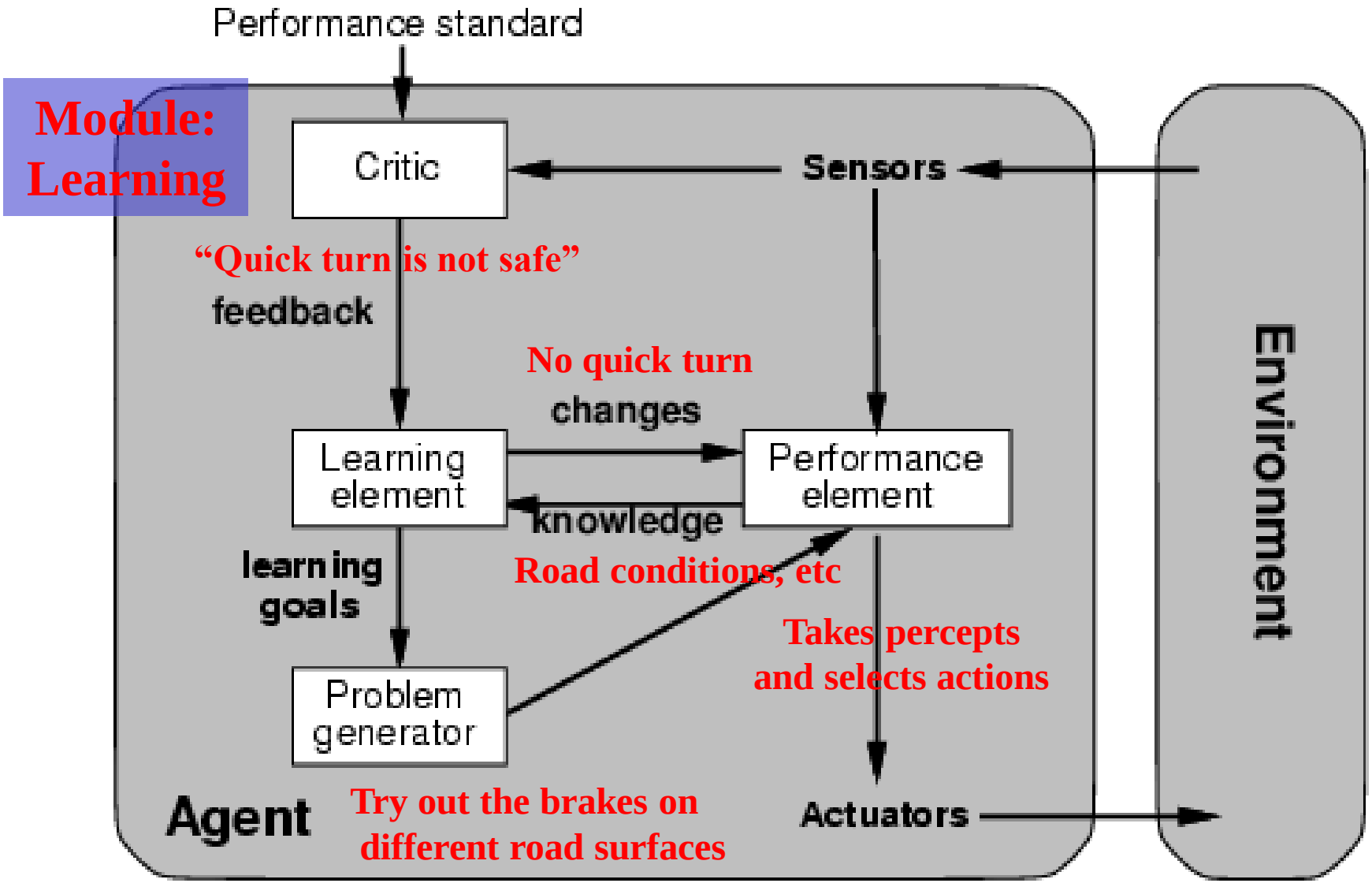
6. Learning Agent

- Performance element
  - selects actions based on percepts, internal state, background knowledge
  - can be one of the previously described agents
- Learning element
  - identifies improvements
- Critic
  - provides feedback about the performance of the agent
  - can be external; sometimes part of the environment
- Problem generator
  - suggests actions
  - required for novel solutions (creativity)

VI) --- Learning agents

Adapt and improve over time

More complicated when agent needs to learn utility information: Reinforcement learning



How the components of agent programs work

- Atomic
- Factored
  - Variable
  - Attribute
  - Value
- Structured
  - Relational database
  - First order logic
  - Probability models

Summary: agent types

- **(1) Table-driven agents**
  - **use a percept sequence/action table in memory to find the next action. They are implemented by a (large) lookup table.**
- **(2) Simple reflex agents**
  - **are based on condition-action rules, implemented with an appropriate production system. They are stateless devices which do not have memory of past world states.**
- **(3) Agents with memory - Model-based reflex agents**
  - **have internal state, which is used to keep track of past states of the world.**
- **(4) Agents with goals – Goal-based agents**
  - **are agents that, in addition to state information, have goal information that describes desirable situations. Agents of this kind take future events into consideration.**
- **(5) Utility-based agents**
  - **base their decisions on classic axiomatic utility theory in order to act rationally.**
- **(6) Learning agents**
  - **they have the ability to improve performance through learning.**

Summary

- An **agent** perceives and acts in an environment, has an architecture and is implemented by an agent program.
- An **ideal agent** always chooses the action which maximizes its expected performance, given percept sequence received so far.
- An **autonomous agent** uses its own experience rather than built-in knowledge of the environment by the designer.
- An **agent program** maps from percept to action & updates its internal state.
  - **Reflex agents** respond immediately to percpets.
  - **Goal-based agents** act in order to achieve their goal(s).